Plan Offero — Parte 7 de 10

Matriz de habilidades, plan de aprendizaje y guía de arranque día a día · Versión 2 · 30 de octubre de 2026 (Wilson en base de datos)

11. Matriz de habilidades y plan de aprendizaje

11.1 Matriz de habilidades

Niveles: 0 = nunca lo ha usado · 1 = lo hace siguiendo instrucciones y con ayuda · 2 = lo hace solo en casos normales · 3 = lo enseña y resuelve problemas. Cada celda es **actual → requerido**. El nivel actual de Iker, Daniel y Wilson sale de tu respuesta («ninguno» tiene experiencia previa en JavaScript, SQL o Git); el de Wilson en Git sube a 1 por su repositorio. El de Diego sale de su experiencia con Next.js, Tailwind, Vercel y Supabase Auth [SUPUESTO en SQL y RLS].

Tema	Diego	Iker	Daniel	Wilson	Por qué ese requerido
Git / GitHub	3 → 3	0 → 2	0 → 2	1 → 2	Todos trabajan con ramas, PR y conflictos
JavaScript moderno	3 → 3	0 → 2	0 → 2	1 → 2	Base de todo lo demás. Wilson ya sabe un poco
React	3 → 3	0 → 2	0 → 2	0 → 2	Cada módulo tiene componentes propios
Next.js App Router	3 → 3	0 → 2	0 → 2	0 → 2	Rutas, Server Actions, servidor y cliente
Tailwind	3 → 3	0 → 1	0 → **3**	0 → 2	Daniel es dueño del diseño base
SQL y PostgreSQL	2 → 3	0 → 2	0 → 2	0 → **3**	Cada uno crea sus tablas; Wilson es el responsable de la base de datos
Supabase Auth	3 → 3	0 → 1	0 → 1	0 → 1	Los demás usan el contrato de Diego, no lo construyen
RLS	2 → 3	0 → **3**	0 → 2	0 → **3**	Iker revisa la seguridad; Wilson revisa todo el SQL
Vercel	3 → 3	0 → 1	0 → **2**	0 → 1	Daniel es respaldo de despliegue

Brechas más grandes: Wilson en SQL y PostgreSQL (0 → 3), Iker en RLS (0 → 3) y Daniel en Tailwind (0 → 3). Las tres necesitan práctica desde el Sprint 0, no solo «justo a tiempo». Diego no tiene brecha técnica: su reto es enseñar y pasarle la base de datos a Wilson sin convertirse en cuello de botella.

11.2 Aprendizaje «justo a tiempo»

- Se aprende lo necesario para la **siguiente historia**, no todo el stack antes de empezar. Cuatro semanas de tutoriales sin construir nada es la forma más común de no terminar.
- Antes de cada tarea nueva, 20–30 minutos con la documentación oficial o la IA sobre ese tema concreto, y luego directo a la tarea.
- Cuando algo se aprende con dolor (un error que costó horas), se anota en `docs/aprendizaje/` en 3–5 líneas: qué pasó, por qué y cómo se resolvió.
- No se recomiendan cursos: la ruta indica temas y orden. Cada uno elige cómo aprenderlos.

11.3 Ruta de aprendizaje por persona

Iker — M7, M6 y revisor de RLS

Cuándo	Qué aprender, en orden (cada tema se apoya en el anterior)
Sprint 0	Git en equipo → JavaScript moderno (const/let, funciones flecha, map y filter, desestructuración, async/await) → componentes de React y props → App Router (carpetas como rutas, servidor y cliente) → SQL básico con JOIN → **RLS con práctica en dev** (USING frente a WITH CHECK, probar con dos cuentas)
Sprints 1–2 (M7)	El contrato de autenticación de Diego → Server Actions con validación en el servidor → políticas de admin con is_admin() → tablas que apuntan a «cualquier cosa» (target_type + target_id)
Sprints 4–6 (M6)	UNIQUE y CHECK → RLS con condiciones sobre otras tablas (cotizaciones aceptadas) → agregaciones (promedio y conteo) y vistas → por qué las vistas no tienen RLS
Enero	Funciones con privilegios elevados (para revisar la de aceptar y la de suspender cuentas)

Daniel — M0, M2, QA y diseño base

Cuándo	Qué aprender, en orden (cada tema se apoya en el anterior)
Sprint 0, semana 1 (M0)	Terminal de PowerShell → Git (clonar, rama, commit, push, PR) → npm y package.json → App Router: page.js y layout.js → JSX frente al HTML que ya sabes → **Tailwind y diseño adaptable (sm:, md:)** → leer el log de compilación de Vercel
Sprint 0, semana 2	JavaScript moderno → React y props → SQL básico (tabla, clave primaria y foránea, UNIQUE, CHECK)
Sprints 1–2 (M2)	RLS (USING y WITH CHECK) → leer datos de Supabase en un componente de servidor → Server Actions y validación en el servidor → rutas dinámicas y parámetros de búsqueda
Sprint 3 en adelante	Cómo escribir casos de prueba y reportar errores (rol de QA) → componentes reutilizables y consistencia visual

Wilson — M3, M5 y responsable de base de datos

Cuándo	Qué aprender, en orden (cada tema se apoya en el anterior)
Sprint 0	**Git en equipo primero** (nunca más .rar) → **PostgreSQL desde cero** en offero-dev: tablas, tipos, clave primaria y foránea, NOT NULL, UNIQUE, CHECK, DEFAULT, SELECT con JOIN → JavaScript moderno (ya sabe un poco) → React y App Router (comparados con Flask)
Sprint 1 (con Diego)	Convención de archivos en supabase/sql/ → RLS para requests (en pareja con Iker) → triggers y funciones (las de Diego en profiles) → cómo revisar un PR con SQL → Server Actions y validación
Sprints 2–4 (lidera la BD)	Transacciones y funciones con privilegios (función de aceptar) → índices básicos → vistas y por qué no tienen RLS → aplicar SQL en offero-prod de forma segura
Sprints 5–7 (M5)	RLS que consulta otras tablas (requests y quotes) → funciones de ayuda en la base (can_message) → agrupar datos para la bandeja
Enero (opcional)	Supabase Realtime, solo si el PO aprueba el tiempo real

Diego — M1, M4, PO y SM

Cuándo	Qué aprender, en orden (cada tema se apoya en el anterior)
Repaso técnico	@supabase/ssr con la versión de Next.js instalada (en Next.js 16 el middleware pasó a llamarse proxy) → funciones con privilegios elevados en PostgreSQL (validar quién llama, fijar search_path)
Facilitación	Cómo dar retroalimentación en revisiones de código sin reescribir el trabajo del otro → cómo facilitar una Review con el profesor y las empresas

11.4 Sesiones de aprendizaje compartido

Quien aprende algo primero se lo explica a los demás. Formato: 30–45 minutos por Discord con pantalla compartida, grabadas si es posible, y un resumen de media página en `docs/aprendizaje/`. Las horas salen de la capacidad de quien enseña (ya incluidas en las tareas).

Cuándo	Tema	Enseña	Para quién
mar 3 nov	Git en equipo: ramas, PR, actualizar con main, conflictos	Diego	Todos (sobre todo Wilson e Iker)
jue 5 nov	Diseño de la base de datos: tablas, relaciones y estados	Diego y **Wilson**	Todos
vie 20 nov	Convenciones de la base de datos y cómo se revisa un PR con SQL	**Wilson**	Todos
mié 11 nov	El contrato de autenticación: usuario en el servidor, proteger páginas y Server Actions	Diego	Todos
lun 16 nov	RLS en la práctica: escribir una política y probarla con dos cuentas	**Iker**	Daniel y Wilson
mié 18 nov	Convenciones de diseño: layout, colores y componentes de Tailwind	**Daniel**	Todos
lun 23 nov	Formularios con Server Actions y validación en el servidor (lo que aprendió en M3)	**Wilson**	Iker y Daniel
mié 25 nov	Leer errores de Supabase y logs de Vercel	Diego	Todos
lun 30 nov	Cómo integrar ReportButton y cómo se modera	**Iker**	Todos
mié 2 dic	La función atómica de aceptar: por qué y cómo se revisa	Diego e Iker	Todos
ene (opcional)	Supabase Realtime	**Wilson**	Todos

Cada uno de los cuatro enseña al menos una vez: así nadie es solo «receptor» y el conocimiento no depende de Diego (factor bus).

12. Guía de arranque: primeras 2 semanas, día a día

Del jueves 29 de octubre al miércoles 11 de noviembre. Los comandos son para **PowerShell en Windows**. Abre una PowerShell nueva después de instalar Node y Git. Lo ya hecho por Diego (repositorio protegido, colaboradores, etiqueta, dos proyectos de Supabase) se marca como hecho.

Jueves 29 de octubre (hoy)

Objetivo: que los cuatro tengan acceso y su computador listo.

Quién	Tareas y pasos	Cómo verificar
Diego	**Hecho:** repositorio protegido, colaboradores invitados, etiqueta prototipo-v0, offero-dev y offero-prod. Pendiente: avisar por WhatsApp que acepten las invitaciones (la de Supabase vence en 24 h) y escribirle al profesor para confirmar la entrega del 29 de enero.	Los tres aparecen en Settings → Collaborators y en el equipo de Supabase

Quién	Tareas y pasos	Cómo verificar

Iker, Daniel, Wilson	1. Aceptar la invitación de GitHub (correo o github.com/notifications) y la de Supabase.
	2. Instalar Node.js LTS: `winget install OpenJS.NodeJS.LTS`
	3. Instalar Git: `winget install Git.Git`
	4. Cerrar y abrir PowerShell. Permitir scripts de npm: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responder S)
	5. Configurar Git con el correo de tu cuenta de GitHub:
	`git config --global user.name "Tu Nombre"`
	`git config --global user.email "tu-correo-de-github"`
	`git config --global init.defaultBranch main`
		`node -v`, `npm -v`, `npx -v` y `git --version` muestran versión, sin el error «scripts deshabilitados».
		`git config --global --list` muestra tu nombre y correo

Viernes 30 de octubre

Objetivo: crear el proyecto Next.js (M0) y dejar Auth configurado.

Quién	Tareas y pasos	Cómo verificar
Daniel (con Wilson)	1. Clonar fuera de OneDrive: `mkdir C:\dev` → `cd C:\dev` → `git clone https://github.com/diegodani2030/App-Offero.git` 2. `cd App-Offero` → `git switch -c chore/m0-proyecto-base` 3. Mover CONTEXTO, EXPOSICION, CHECKLIST y RUTA.md a `docs/`; borrar PROTOTIPOS; commit. 4. En `C:\dev`: `npx create-next-app@latest offero-base` (TypeScript No · ESLint Sí · Tailwind Sí · src/ Sí · App Router Sí · resto por defecto) 5. Copiar sin node_modules ni .git: `robocopy C:\dev\offero-base C:\dev\App-Offero /E /XD node_modules .git` 6. `cd C:\dev\App-Offero` → `npm install` → `npm run dev`	http://localhost:3000 muestra la página inicial de Next.js. `git status` no muestra node_modules
Diego	En offero-dev y offero-prod: Authentication → URL Configuration → Site URL `http://localhost:3000` y agregarla en Redirect URLs. Decidir D-09 (correos de confirmación) y anotarla en la sección 11.	Las dos configuraciones guardadas; D-09 registrada

Iker	Leer completos el documento de contexto y su guía. Instalar extensiones de VS Code:
	`code --install-extension dbaeumer.vscode-eslint`
	`code --install-extension bradlc.vscode-tailwindcss`
	`code --install-extension esbenp.prettier-vscode`
		Las extensiones aparecen en la pestaña Extensiones

Wilson	Leer el documento de contexto y su guía (tabla Flask → Next.js). Instalar las mismas extensiones. Acompañar a Daniel en los pasos 4 a 6.	Puede explicar qué hay en src/app

Sábado 31 de octubre

Objetivo: tener el proyecto base fusionado en main.

Quién	Tareas y pasos	Cómo verificar
Daniel	`git add .` → `git commit -m "chore(m0): crea proyecto Next.js con Tailwind"` → `git push -u origin chore/m0-proyecto-base` Abrir el PR en GitHub con descripción, captura y checklist. Borrar `C:\dev\offero-base`.	El PR existe y pide 1 aprobación

Quién	Tareas y pasos	Cómo verificar
Wilson	Revisar el PR de Daniel como segundo revisor: leer los archivos y dejar al menos un comentario o pregunta.	Su comentario aparece en el PR
Diego	Revisar y fusionar con **Create a merge commit**.	main tiene el proyecto Next.js
Iker	Clonar en C:\dev, `npm install` y `npm run dev` con main actualizado.	localhost:3000 funciona en su computador

Domingo 1 de noviembre

Objetivo: primer despliegue en Vercel.

Quién	Tareas y pasos	Cómo verificar
Diego (con Daniel, 30 min)	1. En vercel.com: Add New → Project → importar App-Offero (framework Next.js detectado). 2. Environment Variables: URL y clave pública de **offero-prod** en Production; las de **offero-dev** en Preview. 3. Deploy. 4. Agregar la URL de producción en Redirect URLs de offero-prod y el patrón de Preview en offero-dev.	La URL de producción carga la portada sin errores en la consola
Todos	Abrir la URL en el celular y avisar en Discord.	Los cuatro la ven

Lunes 2 de noviembre (festivo, día liviano)

Objetivo: que todos tengan variables de entorno y empezar M1.

Quién	Tareas y pasos	Cómo verificar
Iker, Daniel, Wilson	`git switch main` → `git pull` → `npm install` → `Copy-Item .env.example .env.local` y llenar con la URL y la clave pública de dev que Diego envía por mensaje privado.	`git status` NO muestra .env.local; `npm run dev` arranca sin errores
Diego + Iker (1,5 h)	Pareja 1 de M1: SQL de profiles, trigger de registro y vista public_profiles en offero-dev, en un archivo de `supabase/sql/`.	Registrar un usuario de prueba crea su perfil con rol client

Martes 3 de noviembre

Objetivo: terminar la base de M1 y el primer PR de Wilson.

Quién	Tareas y pasos	Cómo verificar
Diego + Iker (2,5 h)	Pareja 2: funciones is_admin() y get_my_role() y políticas de profiles. Probar con dos cuentas de prueba (`tunombre+cliente1@gmail.com`, `tunombre+proveedor1@gmail.com`).	Cada cuenta ve solo su perfil; un registro con rol admin queda como client
Daniel	Rama `feat/m0-portada`: portada, Header y Footer, idioma es y metadatos.	Se ve bien en vista de celular (herramientas del navegador)
Wilson	Primer PR: `git switch main` → `git pull` → `git switch -c docs/m0-integrante-wilson` → agregar su fila en la tabla de integrantes del documento → commit → push → PR.	PR revisado y fusionado, sin .rar ni node_modules
Todos	Sesión de 30 min con Diego: Git en equipo (actualizar la rama con main y resolver un conflicto de práctica).	Cada uno resolvió un conflicto de práctica

Miércoles 4 de noviembre

Objetivo: tablero listo y archivos base del repositorio.

Quién	Tareas y pasos	Cómo verificar
Diego	GitHub → Projects → New project → Board «Offero». Columnas: Backlog, Listo, En curso, En revisión, Hecho. Etiquetas por módulo (m0…m7), tipo (feat, fix, docs, seguridad) y prioridad (must, should, could). Milestones Sprint 0 a Sprint 8. Crear los issues del Sprint 0 y 1 (Parte 6).	El tablero muestra las tareas T0 y T1 en su columna
Wilson	Plantillas: **ya hechas** (PR #113). En su lugar: práctica de PostgreSQL en el SQL Editor de offero-dev (tabla de práctica con clave foránea y CHECK; luego se borra).	Puede explicar clave primaria, clave foránea y CHECK
Daniel	README con instalación en Windows, `.env.example` con solo nombres de variables y excepción para él en `.gitignore`.	`.env.example` está en GitHub; `.env.local` no
Iker	Práctica de RLS en dev: tabla de práctica con dos políticas, probadas con dos cuentas; luego se borra.	Puede explicar USING frente a WITH CHECK

Jueves 5 de noviembre

Objetivo: diseñar juntos la base de datos.

Quién	Tareas y pasos	Cómo verificar
Todos (2 h, facilitan Diego y Wilson)	Agenda: 20 min repaso de tablas del documento · 40 min relaciones (quién apunta a quién) · 30 min estados acordados y is_hidden · 20 min quién crea qué y cuándo (Parte 3, 6.1) · 10 min dudas. Resultado: diagrama o tabla en `docs/esquema.md`.	Cada uno explica cómo se conecta su tabla con las demás
Daniel	Probar el flujo de Preview con un PR pequeño (texto del pie).	El PR muestra el enlace de Preview. Si dice «Deployment Blocked», avisar a Diego

Viernes 6 de noviembre

Objetivo: cerrar pendientes del Sprint 0.

Quién	Tareas y pasos	Cómo verificar
Diego	Clientes de Supabase para servidor y navegador y archivo de sesión (inicio).	La rama compila sin errores
Iker	Checklist de revisión de RLS en `docs/checklist-rls.md`.	Diego la revisa y aprueba
Wilson	Práctica guiada en una rama propia que no se fusiona: un componente con props y una lista con map.	Corre en localhost
Daniel	Ajustes de la portada según la revisión.	PR aprobado

Sábado 7 de noviembre

Objetivo: M0 terminado y todo listo para planificar.

Quién	Tareas y pasos	Cómo verificar
Daniel	Actualizar el documento: enlaces del repositorio y la app (sección 1), M0 Terminado y Publicado: Sí (sección 8). Revisar la Definition of Done de M0.	Checklist de M0 completa

Quién	Tareas y pasos	Cómo verificar
Wilson + Diego (20 min)	Revisión: crear rama, commit, PR y actualizar con main sin ayuda.	Lo hace solo
Diego	Preparar la Planning: backlog ordenado; historias del Sprint 1 con la Definition of Ready; lista de categorías reales para Daniel.	Issues del Sprint 1 en «Listo»

Domingo 8 de noviembre

Objetivo: cerrar el Sprint 0 y planificar el Sprint 1 (agendas completas en la Parte 8).

Quién	Tareas y pasos	Cómo verificar
Todos (2 h en total)	Review del Sprint 0 (30 min): abrir la URL, mostrar un PR con Preview y la regla de main. Retrospectiva (30 min, facilita **Iker**). Sprint Planning del Sprint 1 (1 h, facilita Diego): aprobar los acuerdos de trabajo, confirmar el Sprint Goal y que cada uno tome sus tareas.	Sprint Goal escrito en el milestone; cada tarea T1 con responsable y fecha; acuerdos aprobados en `docs/`

Lunes 9 de noviembre

Objetivo: arrancar el Sprint 1.

Quién	Tareas y pasos	Cómo verificar
Todos	Primera Daily asíncrona en #daily (facilita Iker este sprint): qué hice, qué haré, qué me bloquea.	Los cuatro escribieron antes de las 10 p. m.
Daniel	SQL de categories en un archivo `supabase/sql/20261109_HHMM_categories.sql`, aplicarlo en el SQL Editor de offero-dev y avisar en #prs.	Una cuenta sin sesión ve las categorías; una no admin no puede crear
Diego	Terminar clientes y archivo de sesión.	La sesión sobrevive a recargas

Martes 10 de noviembre

Objetivo: M1 núcleo: el equipo ya puede usar la sesión.

Quién	Tareas y pasos	Cómo verificar
Diego	Patrón de protección por rol y sección «Autenticación» del README. Anunciar en Discord: «M1 núcleo listo».	Sin sesión, /perfil redirige a /login
Wilson + Iker	SQL de requests con sus políticas, en pareja.	Un proveedor no puede crear solicitudes
Iker	Proteger /administrador con el patrón.	Una cuenta no admin es redirigida
Daniel	Archivo con las categorías reales; Diego lo aplica en prod.	Las categorías aparecen en dev y prod

Miércoles 11 de noviembre

Objetivo: que todos sepan usar el contrato de autenticación.

Quién	Tareas y pasos	Cómo verificar
Todos (45 min)	Sesión de Diego: obtener el usuario y su perfil en el servidor, proteger una página y una Server Action, usar get_my_role() en una política.	Cada uno protege una página de práctica

Quién	Tareas y pasos	Cómo verificar
Diego	Registro con rol, inicio y cierre de sesión (sigue hasta el sábado).	—
Iker	SQL de reports con RLS.	Un usuario no puede leer reportes de otros
Daniel	SQL de businesses con RLS (sigue el jueves).	—

> **Si un día se atrasa:** no se recupera trabajando de noche. Se aplica la regla de atasco (Parte 3, 7.6), se avisa en la Daily y el resto del sprint se reacomoda en la siguiente Daily. Lo importante de estas dos semanas, en orden: 1) todos con acceso y computador listo, 2) M0 publicado, 3) M1 núcleo el martes 10.